BioKEA BugPicker GitHub Push Guide

A safe, repeatable procedure for publishing BugPicker code and selected active models

Repository: `/home/biokea-06/.openpnp2/scripts/BugPicker`
Remote: `https://github.com/BioKEA/BugPicker`
Default branch: `main`

> **Primary rule:** do not begin with `git add .` or `git add -A`. This working directory contains local labels, reports, model backups, candidate models, and machine calibration. Stage an explicit allowlist and inspect it before committing.

1. What Git Is Doing

Git records snapshots in three steps:

1. **Working tree:** files currently on this computer.
2. **Staging area:** files selected for the next snapshot with `git add`.
3. **Commit:** a named local snapshot created with `git commit`.

`git push` sends committed snapshots to GitHub. It does not send ordinary unstaged files.

2. Open a Terminal and Enter the Repository

```
cd /home/biokea-06/.openpnp2/scripts/BugPicker
pwd
git status --short
git branch --show-current
git remote -v
```

Expected branch: `main`. Expected remote: `origin` pointing to the BioKEA BugPicker GitHub project.

3. Decide What Belongs on GitHub

Publish	Keep local
Python and JavaScript source	Raw QA labels and captured images
Runner scripts	Training ledgers and generated reports
`requirements.txt`, README, START_HERE	Candidate and backup model files
Spreadsheet template	Python caches and virtual environments
Selected active production models	Machine-specific calibration, unless intentionally shared

> **Recommended policy for this project:** publish code plus the active production models. This lets another BugPicker installation run without retraining, while keeping the full training history and local images off GitHub.

4. Configure `.gitignore`

Open the file with the installed terminal editor:

```
nano .gitignore
```

After editing, press `Ctrl+O`, press `Enter` to confirm the filename, then press `Ctrl+X` to exit. Visual Studio Code is also installed, so `code .gitignore` is an alternative when a graphical editor is preferred.

Add these rules below any existing rules:

```
# Python and local environments
__pycache__/
*.py[cod]
.venv/

# Editor and operating-system files
.DS_Store
*~

# Local backups
*.bak
*.bak-*
*.backup_*

# Generated training state and reports
Data/model_update_reports/
Data/taxonomy_feedback/
Data/classifier_feedback/debris_training_ledger.jsonl
Data/classifier_feedback/retrain_report.csv
Data/insect_debris_classifier_candidate.pt
Data/taxonomy_classifier_candidate.pt

# QA labels, reports, candidates, and backups
Data/qa_feedback/

# Local machine calibration
multitray_calibration.json
training_tray_calibration.json
```

> **Important:** `.gitignore` only affects untracked files. If a file is already tracked, Git will continue showing its modifications. Do not use `git rm` without `--cached`, because plain `git rm` also deletes the local file.

Stop tracking local-only files without deleting them

Use this only when the project has agreed that these tracked files should disappear from GitHub:

```
git rm --cached Data/classifier_feedback/insect_debris_feedback.jsonl
git rm --cached multitray_calibration.json
git rm --cached training_tray_calibration.json
```

The files remain on this computer. Their removal from repository tracking becomes part of the next commit.

Active QA models are inside an ignored directory

If active QA models should be published, force-add only the two production files:

```
git add -f Data/qa_feedback/models/nozzle_qa_classifier.pt
git add -f Data/qa_feedback/models/well_qa_classifier.pt
```

Do not force-add the candidate or backup files.

5. Synchronize Before Creating the Commit

First verify that no updater or scan is currently writing files. Then fetch GitHub changes:

```
git fetch origin
git status -sb
```

If the first line says the local branch is behind `origin/main`, integrate those changes before committing:

```
git pull --rebase origin main
```

> If Git reports a conflict, stop and resolve it deliberately. Do not run `git reset --hard`; that can destroy local work.

6. Stage the BugPicker Code Explicitly

Stage source, launchers, documentation, dependencies, and the spreadsheet template:

```
git add .gitignore
git add -- '*.py' '*.js'
git add README.md START_HERE.txt LICENSE requirements.txt
git add Data/plate_spreadsheet_template.xlsx
```

The quoted globs select files in the repository root and avoid shell surprises. Backup names ending in `.bak-...` do not match `*.js`.

Stage selected active production models

```
git add Data/insect_debris_classifier.pt
git add Data/taxonomy_classifier.pt
git add -f Data/qa_feedback/models/nozzle_qa_classifier.pt
git add -f Data/qa_feedback/models/well_qa_classifier.pt
```

If models should not be distributed, skip this section. Model files increase clone and repository size because Git stores new versions over time.

7. Inspect Exactly What Will Be Committed

```
git status
git diff --cached --stat
git diff --cached --name-status
git diff --cached --check
```

Confirm that the staged list does not contain:

- `candidate.pt` or `backup_...` files

- `Data/model_update_reports/`
- QA labels, raw images, or embedding caches
- Personal credentials or tokens
- Unexpected calibration changes

Remove a mistaken file from staging

```
git restore --staged path/to/file
```

This removes the file from the next commit without deleting the local file.

Review source changes in detail

```
git diff --cached -- '*.py' '*.js' '*.txt' '*.md'
```

8. Run Basic Checks

```
python3 -m py_compile -- *.py
git diff --cached --check
```

OpenPnP JavaScript cannot be fully hardware-tested from a terminal. Run relevant OpenPnP workflows carefully after pulling the code onto another machine.

9. Create the Commit

Verify your Git identity:

```
git config user.name
git config user.email
```

This computer is currently configured as `biokea-austin` and `austin@biokea.ai`. To change it for this repository:

```
git config user.name "Your Name"
git config user.email "your-email@example.com"
```

Create the commit:

```
git commit -m "Add QA training, taxonomy, and plating safeguards"
```

Verify it:

```
git show --stat --oneline HEAD
git status -sb
```

10. Push to GitHub

```
git push origin main
```

If authentication is requested, use the organization-approved GitHub method. GitHub does not accept an account password for HTTPS Git operations; typical options are a credential manager, browser authorization, or a personal access token.

> **Successful result:** the command reports that `main` was updated, and `git status -sb` no longer says the local branch is ahead of `origin/main`.

11. Verify on GitHub

1. Open the BioKEA BugPicker repository in a browser.
2. Confirm the latest commit message and timestamp.
3. Open several changed source files and verify the expected content.
4. Confirm no local reports, labels, backups, or credentials were published.

12. Routine Updates Later

```
cd /home/biokea-06/.openpnp2/scripts/BugPicker
git fetch origin
git status -sb
git pull --rebase origin main

# Stage only intended files
git add path/to/changed/source.py path/to/changed/runner.js

git diff --cached --check
git diff --cached --stat
git commit -m "Describe the change"
git push origin main
```

13. Troubleshooting

Push rejected because GitHub has newer commits

```
git pull --rebase origin main
git push origin main
```

A large or private file is staged but not committed

```
git restore --staged path/to/file
```

A local-only file is already tracked

```
git rm --cached path/to/file
git add .gitignore
git commit -m "Stop tracking local generated data"
```

GitHub rejects a file larger than 100 MB

Remove it from staging or the unpushed commit. Do not repeatedly commit the same large file. Large production artifacts should use Git LFS or a release/artifact store chosen by BioKEA.

Undo the most recent commit before pushing, while keeping the files

```
git reset --soft HEAD~1
```

Use this only for a commit that has not been pushed. Review the staged files again afterward.

Check whether a file is ignored

```
git check-ignore -v path/to/file
```

Final Checklist

- The scan/model updater is not writing files.
- `git fetch origin` has been run.
- The staged file list contains only intentional project assets.
- No candidate models, backups, reports, raw labels, tokens, or private data are staged.
- Basic checks pass.
- The commit message describes the release.
- The pushed commit is visible and correct on GitHub.
